

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Aryan Arun	Reg. No.:	192411157
Section / Batch:	BE CSE / 2024	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Code of Conduct

I _____Aryan Arun (192411157)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____Aryan Arun_____

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CSA0624 – DESIGN AND ANALYSIS OF ALGORITHMS

Industry Problem Solving Task

NAME: ARYAN ARUN

REG:NO: 192411157

Q1. Knapsack — Industry Resource Allocation

Problem Analysis

A cloud service provider has a fixed server memory capacity (100 GB) and must decide which client applications to host to maximize total revenue. Each application requires a fixed amount of memory and generates a fixed revenue once running, and it either runs in full or not at all — it cannot be partially allocated memory. This 'take it completely or leave it' constraint is precisely the defining feature of the 0/1 Knapsack Problem: each item (application) has a weight (memory requirement) and a value (revenue), and the goal is to select a subset of items whose total weight does not exceed the capacity while maximizing total value.

Solution Design (Dynamic Programming)

Let there be n applications, each with memory requirement $w[i]$ and revenue $v[i]$, and total capacity $W = 100$ GB. Define $dp[i][w]$ as the maximum revenue obtainable using only the first i applications with total memory not exceeding w . The recurrence relation is:

- $dp[i][w] = dp[i-1][w]$, if $w[i] > w$ (application i cannot fit)
- $dp[i][w] = \max(dp[i-1][w], v[i] + dp[i-1][w - w[i]])$, otherwise

The base case is $dp[0][w] = 0$ for all w (no applications considered means zero revenue). The final answer is $dp[n][W]$. The actual set of selected applications is recovered by backtracking through the table: at each step, if $dp[i][w] \neq dp[i-1][w]$, application i was included.

Why Dynamic Programming is the Right Approach

A brute-force approach would examine all 2^n possible subsets of applications, which becomes computationally infeasible as the number of client applications grows. Dynamic programming avoids this by exploiting two key properties of the problem:

- **Optimal Substructure:** the optimal allocation for capacity w using i applications is built directly from the optimal allocation using $i-1$ applications at capacity w or $w - w[i]$. Solving the smaller subproblems optimally guarantees the larger solution is optimal.
- **Overlapping Subproblems:** many different decision paths (different subsets of applications considered in different orders) end up needing the value of the same (i, w) pair. For example, $dp[3][60]$ might be needed both when deciding on application 4 and application 5. Naive recursion recomputes this repeatedly; DP computes it once and reuses it, reducing the algorithm from exponential to polynomial time.

Suggested Tools / Platforms for Implementation

- Python (NumPy for vectorized DP table operations) or Java/C++ for performance-critical allocation engines.
- Apache Spark or a distributed job scheduler if the number of client applications is very large and allocation needs to run across a cluster.
- Kubernetes resource quotas / schedulers as the real deployment target once the optimal allocation is computed, since Kubernetes already manages memory limits per pod.
- A monitoring dashboard (Grafana/Prometheus) to feed real, live revenue and memory-usage data back into the model periodically.

Complexity Analysis

- **Time Complexity:** $O(n * W)$, where n is the number of applications and W is the total capacity (100 GB in this scenario). This is pseudo-polynomial — efficient for practical capacity values, though it grows if capacity is expressed with very large numbers.
- **Space Complexity:** $O(n * W)$ for the full 2-D table; this can be reduced to $O(W)$ using a single rolling 1-D array if only the maximum revenue (not the selected items) is required.

Industry Interpretation

For a cloud provider, this DP approach guarantees the mathematically optimal revenue for a given memory budget, which is valuable because memory is a hard physical constraint that cannot be exceeded. In practice, capacity (100 GB) is a modest number, so the pseudo-polynomial $O(n \cdot W)$ runtime is fast enough to re-run this allocation every time client demand changes, supporting near-real-time re-optimization of server allocation decisions.

Python Implementation

```
# 0/1 Knapsack - Cloud Server Memory Allocation
# Capacity = 100 GB total server memory
# Each client application has a memory requirement (weight) and revenue (value)

def knapsack_01(capacity, weights, values, names):
    n = len(weights)
    # dp[i][w] = max revenue using first i apps with capacity w
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(dp[i - 1][w], values[i - 1] + dp[i - 1][w - weights[i - 1]])
            else:
                dp[i][w] = dp[i - 1][w]

    # Backtrack to find which apps were selected
    selected = []
    w = capacity
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i - 1][w]:
            selected.append(names[i - 1])
            w -= weights[i - 1]

    selected.reverse()
    return dp[n][capacity], selected, dp

# Client applications: (name, memory required in GB, revenue in $/hr)
names = ["App-A", "App-B", "App-C", "App-D", "App-E", "App-F"]
weights = [20, 30, 10, 40, 25, 15]
values = [120, 180, 40, 200, 150, 70]
capacity = 100

max_revenue, selected_apps, dp_table = knapsack_01(capacity, weights, values, names)

print("Cloud Server Memory Allocation - 0/1 Knapsack")
print("Total available memory capacity = %d GB" % capacity)
print()
print(f"{'App':<8}{'Memory(GB)':<12}{'Revenue($/hr)'}")
for n_, w_, v_ in zip(names, weights, values):
    print(f"{n_:<8}{w_:<12}{v_}")

print()
print("Applications selected for allocation:", selected_apps)
print("Total memory used:", sum(weights[names.index(a)] for a in selected_apps), "GB")
print("Maximum total revenue achievable = $", max_revenue, "/hr")

print()
print("Sample of DP table (rows = apps considered 0..n, columns = capacity 0..100, showing every 20 GB):")
header = "App#\Cap " + "".join(f"{c:>6}" for c in range(0, capacity + 1, 20))
```

```

print(header)
for i in range(len(dp_table)):
    row = f"{i:<9}" + "".join(f"{dp_table[i][c]:>6}" for c in range(0, capacity + 1, 20))
    print(row)

```

Output

Cloud Server Memory Allocation - 0/1 Knapsack
 Total available memory capacity = 100 GB

App	Memory(GB)	Revenue(\$/hr)
App-A	20	120
App-B	30	180
App-C	10	40
App-D	40	200
App-E	25	150
App-F	15	70

Applications selected for allocation: ['App-A', 'App-B', 'App-C', 'App-E', 'App-F']
 Total memory used: 100 GB
 Maximum total revenue achievable = \$ 560 /hr

Sample of DP table (rows = apps considered 0..n, columns = capacity 0..100, showing every 20 GB):

App#\Cap	0	20	40	60	80	100
0	0	0	0	0	0	0
1	0	120	120	120	120	120
2	0	120	180	300	300	300
3	0	120	220	340	340	340
4	0	120	220	340	420	540
5	0	120	220	340	450	540
6	0	120	220	340	450	560

Q2. TSP — Logistics Optimization (Held-Karp)

Problem Analysis

A delivery company must visit 15 cities exactly once each and return to its depot, minimizing total travel cost. This is the classical Travelling Salesman Problem (TSP): find the minimum-cost Hamiltonian cycle over a set of cities. Because delivery cost (fuel, driver time) scales directly with distance travelled, solving TSP optimally directly reduces operating cost.

Solution Design — State Representation and Recurrence

The Held-Karp algorithm solves TSP exactly using dynamic programming over bitmask states. Let the depot be city 0. A state is defined as (S, j) , where S is a bitmask representing the subset of visited cities (excluding the depot) and j is the last city visited in that subset. $dp[S][j]$ represents the minimum cost of a path that starts at the depot, visits exactly the cities in S , and ends at city j .

- Base case: $dp[\{j\}][j] = \text{dist}(0, j)$, for each city $j \neq 0$.
- Recurrence: $dp[S][j] = \min \text{ over all } k \text{ in } S, k \neq j, \text{ of } (dp[S - \{j\}][k] + \text{dist}(k, j))$.
- Final answer: $\min \text{ over all } j \text{ of } (dp[\text{Full Set}][j] + \text{dist}(j, 0))$, closing the tour back at the depot.

Each state (S, j) is computed once and reused whenever a larger subset's recurrence needs it — this is the same overlapping-subproblems principle as Knapsack, applied here over subsets of cities rather than capacities.

Feasibility in Real-World Logistics

Held-Karp reduces the brute-force factorial time of $(n-1)!/2$ permutations down to $O(2^n * n^2)$, which is a dramatic improvement, and it guarantees a truly optimal tour rather than an approximation. For the 15-city case in this task it remains computationally practical, as shown by the runtime captured below. However, the exponential term 2^n means this exact approach stops being practical once the number of cities grows into the high 20s or beyond — at that point most logistics companies switch to heuristic or metaheuristic methods (nearest neighbor, 2-opt, simulated annealing, or Genetic Algorithms) that trade a small amount of optimality for large speed gains, or to Integer Linear Programming solvers with cutting planes for near-optimal results at larger scale.

Suggested Tools / Platforms for Implementation

- Python with NumPy for the DP table, or C++ for a faster low-level implementation of Held-Karp on larger fleets.
- Google OR-Tools — an industry-standard routing/optimization library with a dedicated Vehicle Routing Problem (VRP) solver used widely in logistics.
- Gurobi or CPLEX (commercial ILP solvers) for exact or near-exact solutions at larger scale using integer programming formulations of TSP.
- GIS/mapping APIs (Google Maps Distance Matrix API, OSRM) to generate real road-network travel-time matrices rather than straight-line distances.

Scalability Analysis

- Time Complexity: $O(2^n * n^2)$. For $n = 15$, this is about $2^{14} * 14^2 \approx 3.2$ million operations — fast. For $n = 20$, it grows to roughly $2^{19} * 19^2 \approx 189$ million operations — still feasible but noticeably slower. For $n = 25$ or above, the state space (2^n) becomes tens of millions to billions of entries, making Held-Karp impractical on typical hardware.
- Space Complexity: $O(2^n * n)$, since a DP value must be stored for every (subset, last-city) pair. This memory requirement, not just the time, is usually what makes Held-Karp infeasible first as n grows — for $n = 25$ the table would need billions of entries.
- Practical limit: Held-Karp is generally considered usable only up to roughly 20-23 cities on standard hardware; beyond that, approximate methods are required.

Industry Interpretation

For a fleet visiting 15 cities per route, Held-Karp is an excellent fit — it delivers a provably optimal route with a runtime of well under a second, as shown below, directly minimizing fuel and driver-hours cost. If the company's routes grow beyond roughly 20-25 stops (e.g. dense urban delivery routes), the exact approach should be replaced with OR-Tools' heuristics or a metaheuristic to keep route planning fast enough for daily operations.

Python Implementation

```
# Travelling Salesman Problem - Held-Karp Dynamic Programming Algorithm
# Logistics: a delivery vehicle must visit 15 cities exactly once and return to depot

import random
import itertools
import time

def generate_distance_matrix(n, seed=42):
    random.seed(seed)
    # Random coordinates for n cities on a 100x100 grid (depot = city 0)
    coords = [(random.randint(0, 100), random.randint(0, 100)) for _ in range(n)]
    dist = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i != j:
                dx = coords[i][0] - coords[j][0]
                dy = coords[i][1] - coords[j][1]
                dist[i][j] = round((dx ** 2 + dy ** 2) ** 0.5, 2)
    return dist, coords

def held_karp(dist):
    n = len(dist)
    # dp[(mask, j)] = minimum cost to reach subset 'mask' of cities, ending at city j, starting
    # from city 0
    dp = {}
    parent = {}

    for k in range(1, n):
        dp[(1 << k, k)] = dist[0][k]

    for subset_size in range(2, n):
        for subset in itertools.combinations(range(1, n), subset_size):
            mask = 0
            for city in subset:
                mask |= (1 << city)
            for j in subset:
                prev_mask = mask & ~(1 << j)
                best_cost = float('inf')
                best_k = -1
                for k in subset:
                    if k == j:
                        continue
                    if (prev_mask, k) in dp:
                        cost = dp[(prev_mask, k)] + dist[k][j]
                        if cost < best_cost:
                            best_cost = cost
                            best_k = k
                if best_k != -1:
                    dp[(mask, j)] = best_cost
                    parent[(mask, j)] = best_k

    # Close the tour: return to depot (city 0)
```

```

full_mask = (1 << n) - 2 # all cities except city 0
best_cost = float('inf')
best_last = -1
for k in range(1, n):
    if (full_mask, k) in dp:
        cost = dp[(full_mask, k)] + dist[k][0]
        if cost < best_cost:
            best_cost = cost
            best_last = k

# Reconstruct path
path = [0]
mask = full_mask
last = best_last
route = []
while last != -1:
    route.append(last)
    if (mask, last) not in parent:
        break
    new_last = parent[(mask, last)]
    mask = mask & ~(1 << last)
    last = new_last
route.reverse()
path.extend(route)
path.append(0)

return best_cost, path

N_CITIES = 15
dist_matrix, coords = generate_distance_matrix(N_CITIES)

print(f"Number of cities (including depot) = {N_CITIES}")
print("Depot = City 0")
print()

start = time.time()
min_cost, optimal_path = held_karp(dist_matrix)
elapsed = time.time() - start

print("Optimal tour (city sequence):", " -> ".join(str(c) for c in optimal_path))
print(f"Minimum total travel cost = {min_cost:.2f} distance units")
print(f"Held-Karp computation time = {elapsed:.4f} seconds")
print(f"Number of DP states explored (approx) = 2^{N_CITIES-1} * {N_CITIES-1} = {(2**(N_CITIES-1))*(N_CITIES-1):,}")

```

Output

```

Number of cities (including depot) = 15
Depot = City 0

Optimal tour (city sequence): 0 -> 4 -> 11 -> 6 -> 2 -> 3 -> 8 -> 9 -> 10 -> 1 -> 5 -> 13 -> 14
-> 7 -> 12 -> 0
Minimum total travel cost = 406.68 distance units
Held-Karp computation time = 0.4825 seconds
Number of DP states explored (approx) = 2^14 * 14 = 229,376

```

Q3. Floyd–Warshall — Network Routing

Problem Analysis

A telecom company needs the shortest (lowest-latency) path between every pair of routers in its network, not just from a single source. This all-pairs requirement is exactly what the Floyd-Warshall algorithm is designed to compute efficiently in a single unified pass, making it well suited to precomputing routing tables for an entire network.

Solution Design — Role of Intermediate Nodes

Floyd-Warshall works by progressively allowing each router, one at a time, to act as a possible intermediate hop between every pair of routers. Let $\text{dist}[i][j]$ be the shortest known distance from router i to router j . For each candidate intermediate router k (from 1 to n), and for every pair (i, j) , the algorithm checks whether routing through k is cheaper than the current best known path:

- Recurrence: $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$

After considering all n routers as possible intermediates, $\text{dist}[i][j]$ holds the true shortest path cost between every pair, because any shortest path can be decomposed into sub-paths through a sequence of intermediate nodes, and the algorithm has tested every such node as a potential improvement.

Suggested Tools / Technologies for Implementation

- Python/NumPy or C++ for computing the matrix, especially if latency needs to be recalculated frequently as link weights change.
- Graph libraries such as NetworkX (Python) or the Boost Graph Library (C++) which include optimized Floyd-Warshall / all-pairs shortest path implementations.
- SDN controllers (e.g. OpenDaylight, ONOS) which can consume a computed routing/latency matrix to program flow tables in real routers.
- A time-series database (InfluxDB/Prometheus) to track live link latencies feeding into a periodically-recomputed cost matrix.

Complexity Analysis

- Time Complexity: $O(n^3)$, where n is the number of routers — three nested loops over all routers for the intermediate node, source, and destination.
- Space Complexity: $O(n^2)$ to store the distance matrix (and optionally another $O(n^2)$ matrix to reconstruct actual paths, as done in the implementation below).
- Because of the cubic growth, Floyd-Warshall is best suited to small-to-medium sized networks (up to a few hundred routers); very large ISP-scale networks typically rely on distributed, incremental protocols (e.g. OSPF/Dijkstra per node) instead.

Interpretation — Network Efficiency and Applicability

The resulting all-pairs matrix lets the telecom operator instantly determine both the lowest-latency route and its cost between any two routers, without having to run a separate shortest-path computation for each pair. This is valuable for pre-provisioning backup/alternate paths and for capacity planning, since it reveals which links are heavily relied upon as intermediate hops across many router pairs. It works best for a fixed or infrequently-changing core network topology; if link weights change constantly, a full $O(n^3)$ recomputation on every change may be too slow for very large n , favoring incremental or per-node approaches instead.

Python Implementation

```
# Floyd-Warshall Algorithm - Telecom Network Routing
# Computes shortest paths between all pairs of routers
```

```

INF = float('inf')

def floyd_warshall(graph):
    n = len(graph)
    dist = [row[:] for row in graph]          # cost matrix
    next_node = [[None] * n for _ in range(n)]

    for i in range(n):
        for j in range(n):
            if i != j and dist[i][j] != INF:
                next_node[i][j] = j

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
                    next_node[i][j] = next_node[i][k]

    return dist, next_node

def reconstruct_path(u, v, next_node):
    if next_node[u][v] is None:
        return None
    path = [u]
    while u != v:
        u = next_node[u][v]
        path.append(u)
    return path

routers = ["R1", "R2", "R3", "R4", "R5", "R6"]
n = len(routers)

# Adjacency matrix (latency in ms). INF = no direct link.
graph = [
    [0, 4, INF, 5, INF, INF],
    [4, 0, 1, INF, 6, INF],
    [INF, 1, 0, 3, 2, INF],
    [5, INF, 3, 0, INF, 2],
    [INF, 6, 2, INF, 0, 1],
    [INF, INF, INF, 2, 1, 0],
]

print("Telecom Router Network - Initial Latency Matrix (ms):")
header = " " + "".join(f"{r:>6}" for r in routers)
print(header)
for i, row in enumerate(graph):
    print(f"{routers[i]:<5}" + "".join(f"({'INF' if c == INF else c):>6}" for c in row))

dist, next_node = floyd_warshall(graph)

print("\nAll-Pairs Shortest Path Matrix After Floyd-Warshall (ms):")
print(header)
for i, row in enumerate(dist):
    print(f"{routers[i]:<5}" + "".join(f"({'INF' if c == INF else c):>6}" for c in row))

print("\nSample shortest paths:")
for (u, v) in [(0, 5), (1, 3), (4, 0)]:
    path = reconstruct_path(u, v, next_node)

```

```
path_names = " -> ".join(routers[p] for p in path) if path else "No path"
print(f"{routers[u]} to {routers[v]}: cost = {dist[u][v]} ms, path = {path_names}")
```

Output

Telecom Router Network - Initial Latency Matrix (ms):

	R1	R2	R3	R4	R5	R6
R1	0	4	INF	5	INF	INF
R2	4	0	1	INF	6	INF
R3	INF	1	0	3	2	INF
R4	5	INF	3	0	INF	2
R5	INF	6	2	INF	0	1
R6	INF	INF	INF	2	1	0

All-Pairs Shortest Path Matrix After Floyd-Warshall (ms):

	R1	R2	R3	R4	R5	R6
R1	0	4	5	5	7	7
R2	4	0	1	4	3	4
R3	5	1	0	3	2	3
R4	5	4	3	0	3	2
R5	7	3	2	3	0	1
R6	7	4	3	2	1	0

Sample shortest paths:

R1 to R6: cost = 7 ms, path = R1 -> R4 -> R6

R2 to R4: cost = 4 ms, path = R2 -> R3 -> R4

R5 to R1: cost = 7 ms, path = R5 -> R3 -> R2 -> R1

Q4. OBST — Database Optimization

Problem Analysis

A database system stores a set of sorted record keys, each with a known probability of being searched for. If the keys are placed in a plain binary search tree without regard to these probabilities, frequently-accessed keys could end up deep in the tree, causing unnecessarily expensive lookups on average. The Optimal Binary Search Tree (OBST) problem asks: given the keys and their access probabilities, how should the BST be structured to minimize the expected number of comparisons per search, i.e., minimize expected query cost?

Solution Design — Cost Function and Recurrence

Let keys $k[0..n-1]$ be sorted, each with access probability $p[i]$. Define $\text{cost}[i][j]$ as the minimum expected search cost of a BST containing only keys $k[i..j]$. Choosing key $k[r]$ (for $i \leq r \leq j$) as the root of that subtree splits the remaining keys into a left subtree $k[i..r-1]$ and a right subtree $k[r+1..j]$. Every node in this subtree is one comparison deeper than it would be at the top, so the total probability of the range, $\text{sum}(p[i..j])$, is added once for that extra depth. The recurrence is:

- $\text{cost}[i][i] = p[i]$ (a single key is its own tree)
- $\text{cost}[i][j] = \min \text{ over } r \text{ in } [i, j] \text{ of } (\text{cost}[i][r-1] + \text{cost}[r+1][j] + \text{sum}(p[i..j]))$

The overall answer is $\text{cost}[0][n-1]$. A parallel $\text{root}[i][j]$ table records which key minimized each range so the actual tree structure can be reconstructed afterward, as done in the implementation below.

How Probabilities Influence Tree Construction

Because every choice of root adds the same fixed range-probability $\text{sum}(p[i..j])$ regardless of which key is chosen, the algorithm is effectively minimizing how deep the high-probability keys sit rather than the tree's raw shape. The recurrence naturally favors placing keys with larger access probabilities closer to the root, since that keeps the more frequently accessed records at a shallow depth (fewer comparisons), while rarely-accessed keys are pushed deeper where the extra comparisons cost less in expectation. In the example below, K40 (probability 0.35, the highest) becomes the root.

Suggested Tools for Implementation

- Python for prototyping the $O(n^3)$ DP and validating tree structure, as shown below.
- C/C++ or Java within the actual database engine's indexing layer, where OBST-style balancing logic can inform or initialize a B-tree/B+-tree index (most real databases use B-trees rather than raw BSTs, but the same probability-weighted-depth principle applies to index design).
- Query logs / APM tools (e.g. `pg_stat_statements` for PostgreSQL, slow query logs) to derive real access-probability estimates for each key from production traffic, rather than assuming them.

Efficiency Analysis

- Time Complexity: $O(n^3)$ using the direct DP formulation ($O(n^2)$ subproblems, each taking $O(n)$ to try every possible root). Knuth's optimization (exploiting the fact that the optimal root index is monotonic) can reduce this to $O(n^2)$.
- Space Complexity: $O(n^2)$ for the cost and root tables.

Interpretation in Database Optimization Context

The computed minimum expected cost directly represents the average number of key comparisons per search across the real access pattern, rather than the worst case or an unweighted average. For the example dataset below, structuring the tree optimally around access probabilities gives an expected cost of 1.8 comparisons per search — placing the most frequently requested record (K40) at the root and keeping the least frequently requested record (K30) deep in the tree. Applied to a real database index, this means query latency is minimized for the traffic pattern that actually occurs, rather than for a hypothetical uniform access pattern.

Python Implementation

```
# Optimal Binary Search Tree - Database Query Optimization
# Keys = database record identifiers (kept sorted), each with a known access (search)
probability

def optimal_bst(keys, freq):
    n = len(keys)
    # cost[i][j] = minimum cost of OBST built from keys[i..j]
    cost = [[0] * n for _ in range(n)]
    root = [[0] * n for _ in range(n)]

    for i in range(n):
        cost[i][i] = freq[i]
        root[i][i] = i

    # prefix sums for quick range-frequency sum (used in the recurrence)
    prefix = [0] * (n + 1)
    for i in range(n):
        prefix[i + 1] = prefix[i] + freq[i]

    def range_sum(i, j):
        return prefix[j + 1] - prefix[i]

    for length in range(2, n + 1):          # chain length
        for i in range(0, n - length + 1):
            j = i + length - 1
            cost[i][j] = float('inf')
            total = range_sum(i, j)
            for r in range(i, j + 1):      # try each key as root
                left = cost[i][r - 1] if r > i else 0
                right = cost[r + 1][j] if r < j else 0
                c = left + right + total
                if c < cost[i][j]:
                    cost[i][j] = c
                    root[i][j] = r

    return cost, root

def build_tree_structure(root, i, j, keys):
    if i > j:
        return None
    r = root[i][j]
    left = build_tree_structure(root, i, r - 1, keys) if r > i else None
    right = build_tree_structure(root, r + 1, j, keys) if r < j else None
    return (keys[r], left, right)

def print_tree(node, prefix="", is_root=True):
    if node is None:
        return
    key, left, right = node
    print(prefix + ("Root: " if is_root else "") + str(key))
    if left:
        print(prefix + "  L-> " + str(left[0]))
    if right:
        print(prefix + "  R-> " + str(right[0]))

# Database records (keys) and their known access probabilities (must sum to 1.0)
```

```

keys = ["K10", "K20", "K30", "K40", "K50"]
freq = [0.10, 0.20, 0.05, 0.35, 0.30] # access probabilities

print("Database Records and Access Probabilities:")
for k, f in zip(keys, freq):
    print(f" {k}: probability = {f}")
print(" Sum of probabilities =", round(sum(freq), 4))
print()

cost, root = optimal_bst(keys, freq)
n = len(keys)

print("Minimum expected number of comparisons (cost) for OBST =", round(cost[0][n - 1], 4))
print()

print("Cost table cost[i][j] (minimum expected search cost for keys[i..j]):")
header = " " + "".join(f"{k:>8}" for k in keys)
print(header)
for i in range(n):
    row = f"{keys[i]:<6}"
    for j in range(n):
        row += f"{{round(cost[i][j], 3) if j >= i else '-'}}:>8}"
    print(row)

print()
print("Root table root[i][j] (which key becomes the root for range i..j):")
print(header)
for i in range(n):
    row = f"{keys[i]:<6}"
    for j in range(n):
        row += f"{{keys[root[i][j]] if j >= i else '-'}}:>8}"
    print(row)

tree = build_tree_structure(root, 0, n - 1, keys)
print()
print("Resulting Optimal BST structure (root, then left/right children of root):")
print_tree(tree)

```

Output

```

Database Records and Access Probabilities:
  K10: probability = 0.1
  K20: probability = 0.2
  K30: probability = 0.05
  K40: probability = 0.35
  K50: probability = 0.3
  Sum of probabilities = 1.0

Minimum expected number of comparisons (cost) for OBST = 1.8

Cost table cost[i][j] (minimum expected search cost for keys[i..j]):
      K10    K20    K30    K40    K50
K10    0.1    0.4    0.5    1.2    1.8
K20    -     0.2    0.3    0.9    1.5
K30    -     -     0.05   0.45   1.05
K40    -     -     -     0.35   0.95
K50    -     -     -     -     0.3

```

Root table root[i][j] (which key becomes the root for range i..j):

	K10	K20	K30	K40	K50
K10	K10	K20	K20	K40	K40
K20	-	K20	K20	K40	K40
K30	-	-	K30	K40	K40
K40	-	-	-	K40	K40
K50	-	-	-	-	K50

Resulting Optimal BST structure (root, then left/right children of root):

Root: K40

L-> K20

R-> K50

Q5. Dijkstra — Navigation System

Problem Analysis

A navigation system must compute the shortest (fastest) route from a user's current location to a destination in real time, over a road network where every road segment has a non-negative travel-time weight. Since all weights are non-negative (travel time cannot be negative), Dijkstra's algorithm is the natural and efficient choice for this single-source shortest-path problem.

Solution Design — Step-by-Step Execution with a Priority Queue

Dijkstra's algorithm maintains a tentative shortest-distance value for every location, initialized to infinity except the source (0), and repeatedly selects the unvisited location with the smallest tentative distance to finalize next:

1. Initialize $\text{dist}[\text{source}] = 0$ and $\text{dist}[\text{all others}] = \text{infinity}$; push (0, source) into a min-priority queue.
2. Pop the location u with the smallest distance from the priority queue; if already finalized, skip it.
3. Mark u as finalized (its shortest distance is now certain).
4. Relax each neighbor v of u : if $\text{dist}[u] + \text{weight}(u, v) < \text{dist}[v]$, update $\text{dist}[v]$ and push (new distance, v) into the priority queue.
5. Repeat steps 2-4 until the priority queue is empty or the destination has been finalized.

The priority queue (implemented as a binary min-heap, e.g. Python's `heapq`) is what makes this efficient: it always gives the algorithm the next-closest unvisited location in $O(\log V)$ time, instead of needing an $O(V)$ linear scan to find the minimum at each step.

Suggested Data Structures and Tools

- Binary min-heap (Python `heapq`, Java `PriorityQueue`, C++ `std::priority_queue`) for the priority queue itself.
- Adjacency list (hash map of lists) to represent the road network, since real-world road graphs are sparse (each intersection connects to only a handful of others).
- Fibonacci heap for very large graphs, since it improves the theoretical time complexity of the decrease-key operation used during relaxation.
- Production-grade routing engines: OSRM, GraphHopper, or Valhalla — open-source engines that implement highly optimized variants of Dijkstra/A* over real map data (OpenStreetMap), typically combined with speed-up techniques such as Contraction Hierarchies for near-instant queries on continent-scale road networks.

Time Complexity — Different Approaches

- Array-based (no heap): $O(V^2)$ — simple but slow for large, sparse road networks.
- Binary heap + adjacency list: $O((V + E) \log V)$ — the standard practical implementation, used in the code below.
- Fibonacci heap + adjacency list: $O(E + V \log V)$ — theoretically optimal, but higher constant-factor overhead often makes binary heaps faster in practice except on very large graphs.
- A* (Dijkstra + a distance heuristic, e.g. straight-line distance to destination): explores far fewer nodes than plain Dijkstra in practice, which is why most real navigation systems use A* or Dijkstra with Contraction Hierarchies rather than plain Dijkstra.

Interpretation in Real-World Navigation Systems

The step-by-step trace below shows the algorithm finalizing locations in increasing order of distance from Home, and the final table gives both the shortest travel time and the actual turn-by-turn route to every location. In a production navigation app, this same core logic runs against a road graph with millions of nodes, which is why real systems layer on A* heuristics and precomputed speed-up structures (Contraction Hierarchies, ALT) on top of Dijkstra's core relaxation idea — the underlying correctness guarantee (always expand the closest unvisited node next) remains exactly what is demonstrated here at small scale.

Python Implementation

```
# Dijkstra's Algorithm - Real-Time Navigation System
# Uses a priority queue (min-heap) to always expand the closest unvisited location

import heapq

def dijkstra(graph, source, locations):
    n = len(locations)
    dist = {loc: float('inf') for loc in locations}
    prev = {loc: None for loc in locations}
    dist[source] = 0

    pq = [(0, source)]          # (distance, node)
    visited = set()
    steps = []

    while pq:
        d, u = heapq.heappop(pq)
        if u in visited:
            continue
        visited.add(u)
        steps.append((u, d))

        for v, weight in graph[u]:
            if v not in visited:
                new_dist = d + weight
                if new_dist < dist[v]:
                    dist[v] = new_dist
                    prev[v] = u
                    heapq.heappush(pq, (new_dist, v))

    return dist, prev, steps

def reconstruct_path(prev, source, target):
    path = []
    node = target
    while node is not None:
        path.append(node)
        if node == source:
            break
        node = prev[node]
    path.reverse()
    return path if path[0] == source else None

# Navigation graph: locations and road travel times (minutes)
locations = ["Home", "Mall", "School", "Office", "Park", "Hospital"]
graph = {
    "Home":      [("Mall", 5), ("School", 10)],
    "Mall":      [("Home", 5), ("School", 3), ("Office", 8)],
    "School":    [("Home", 10), ("Mall", 3), ("Park", 6)],
    "Office":    [("Mall", 8), ("Hospital", 4)],
    "Park":      [("School", 6), ("Hospital", 2)],
    "Hospital":  [("Office", 4), ("Park", 2)],
}

source = "Home"
dist, prev, steps = dijkstra(graph, source, locations)
```

```

print(f"Source location: {source}")
print()
print("Step-by-step execution (location finalized, shortest distance so far):")
for loc, d in steps:
    print(f"  Visit '{loc}' -> shortest distance from {source} = {d} min")

print()
print("Final shortest travel times from Home:")
for loc in locations:
    path = reconstruct_path(prev, source, loc)
    path_str = " -> ".join(path) if path else "unreachable"
    print(f"  {loc:<10}: {dist[loc]:>3} min    route: {path_str}")

```

Output

```

Source location: Home

Step-by-step execution (location finalized, shortest distance so far):
  Visit 'Home' -> shortest distance from Home = 0 min
  Visit 'Mall' -> shortest distance from Home = 5 min
  Visit 'School' -> shortest distance from Home = 8 min
  Visit 'Office' -> shortest distance from Home = 13 min
  Visit 'Park' -> shortest distance from Home = 14 min
  Visit 'Hospital' -> shortest distance from Home = 16 min

Final shortest travel times from Home:
Home      :   0 min    route: Home
Mall      :   5 min    route: Home -> Mall
School    :   8 min    route: Home -> Mall -> School
Office    :  13 min    route: Home -> Mall -> Office
Park      :  14 min    route: Home -> Mall -> School -> Park
Hospital  :  16 min    route: Home -> Mall -> School -> Park -> Hospital

```